

Comenzado el	jueves, 28 de mayo de 2020, 18:33
Estado	Finalizado
Finalizado en	jueves, 28 de mayo de 2020, 20:20
Tiempo empleado	1 hora 46 minutos
Vencido	1 minutos 46 segundos
Calificación	8,20 de un máximo de 10,00 (82%)

Pregunta 1

Finalizado

Puntúa 2,00 sobre 2,00

Marcar pregunta

Realice un programa de Comunicación Bidireccional que permita a un padre recibir un mensaje por teclado y enviarla al hijo el cual debe recibirlo mostrarlo en pantalla y almacenarlo y luego el proceso hijo debe enviar el mismo mensaje al padre el cual debe escribirlo en pantalla.

```
#DEFINE SIZE 512
using namespace std;
int main(){
    pid_t pid;
    int fd1[2],fd2[2],largo;
    char buffer[SIZE];
    pipe(fd1);
    pipe(fd2);
    if(pid=fork()!=0){
        close(fd1[0]);
        close(fd2[1]);
        char mensaje[SIZE];
        cout<<"ingrese mensaje"<<endl;
        cin>>mensaje;
        buffer=mensaje;
        write(fd1[1],buffer,strlen(buffer)+1);
        while(largo=read(fd2[0],buffer,sizeof(buffer))>0){
            cout<<"mensaje recibido"<<endl;
        }
        cout<<"mensaje: "<<buffer<<endl;
        close(fd2[0]);
    }else{
        close(fd1[1]);
        close(fd2[0]);
        while(largo=read(fd1[0],buffer,sizeof(buffer))>0){
            cout<<"mensaje recibido"<<endl;
        }
        cout<<"mensaje: "<<buffer<<endl;
        close(fd2[0]);
        write(fd2[1],buffer,strlen(buffer)+1);
        cout<<"mensaje enviado"<<endl;
        close(fd2[1]);
    }
    waitpid(pid,NULL,0);
    return 0;
}
```

Comentario:

Pregunta 2

Finalizado

Puntúa 2,70 sobre 3,00

Marcar pregunta

Dados los siguientes procesos realizar los diagramas de Gantt para una planificación FCFS, SJF, SRT, RR (Q=2) y Por Prioridad No Apropiativa. Además se debe determinar el tiempo de espera, tiempo promedio de espera, tiempo de servicio e índice de servicio.

Proceso	Ráfaga de CPU	Tiempo de llegada	Prioridad
A	3	0	0
B	5	1	1
C	2	4	0
D	6	5	2
E	8	3	2

Completa aquí la respuesta o sube un archivo con la foto de la misma....

Parcial1_planificacion_BOURLOT - Hoja 1.pdf

Comentario:

RR en el instante que termina A su primer ciclo no entra nadie (E entra en el 3) y en la cola de listos solamente estaba B, por lo que al terminar B vuelve a ejecutarse A (NO E).

Pregunta 3

Incorrecta

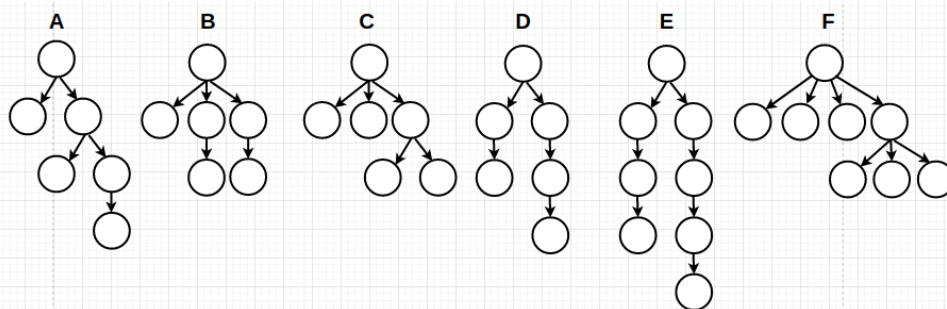
Puntúa 0,00 sobre 1,50

Marcar pregunta

Dado el siguiente código:

```
int main (void)
{
    int num;
    pid_t nuevo;
    nuevo = fork();
    for (num= 0; num<= 2; num++) {
        if (fork()==0) break;
    }
    return 0;
}
```

Seleccione, de la siguiente figura, el árbol de procesos resultante:



Seleccione una:

- ☐ Opción A
- ☒ Opción B ✖
- ☐ Opción C
- ☐ Opción D
- ☐ Opción E
- ☐ Opción F

Respuesta incorrecta.

Pregunta 4

Finalizado

Puntúa 2,00 sobre 2,00

 Marcar pregunta

Considerando el siguiente programa productor-consumidor:

```
#include <iostream>
#include <pthread.h>
#include <semaphore.h>
#include <ctime>
#include <cstdlib>
#include <unistd.h>
#include <stdio.h>
#include <wait.h>

using namespace std ;

const unsigned
    int buffer [5];
    int j = 0;
unsigned producir_dato()
{
    static int contador = 0 ;
    cout << "Producido: " << contador << endl << flush ;
    return contador++ ;
}

void consumir_dato( int dato )
{
    cout << "Dato recibido: " << dato << endl ;
}

void * productor( void * )
{
    for( unsigned i = 0 ; i < 100 ; i++ )
    {
        int elemento = producir_dato();
        cout << "Dato producido: " << elemento << endl;
        buffer[j] = elemento;
        j++;
    }
    return NULL ;
}

void * consumidor( void * )
{
    for( unsigned i = 0 ; i < 100 ; i++ )
    {
        int elemento;
        j--;
        elemento = buffer[j];
        consumir_dato (elemento) ;
    }
    return NULL ;
}

int main()
{

```

Modifique el programa para que sincronice los hilos y controle el acceso a la zona crítica (tipos y métodos utiles pthread_t, pthread_create, pthread_join, sem_t, sem_init, sem_post, sem_wait, pthread_mutex_init, pthread_mutex_lock, pthread_mutex_unlock, etc)

```
#include <iostream>
#include <pthread.h>
```

```

#include <semaphore.h>

#include <ctime>

#include <cstdlib>
#include <unistd.h>
#include <stdio.h>
#include <wait.h>

using namespace std ;

const unsigned
    int buffer [5];
    int j = 0;
    pthread_mutex_t mutex;
    sem_t hay_lugar;
    sem_t hay_datos;

unsigned producir_dato()
{
    pthread_mutex_lock(&mutex);
    static int contador = 0 ;
    cout << "Producido: " << contador << endl << flush ;
    pthread_mutex_unlock(&mutex);
    return contador++ ;
}

void consumir_dato( int dato )
{
    pthread_mutex_lock(&mutex);
    cout << "Dato recibido: " << dato << endl ;
    pthread_mutex_unlock(&mutex);
}

void * productor( void * )
{
    for( unsigned i = 0 ; i < 100 ; i++ )
    {
        sem_wait(&hay_lugar);
        int elemento = producir_dato();
        cout << "Dato producido: " << elemento << endl;
        buffer[j] = elemento;
        j++;
        sem_post(&hay_datos);
    }
    return NULL ;
}

void * consumidor( void * )
{
    for( unsigned i = 0 ; i < 100 ; i++)
    {
        sem_wait(&hay_datos);
        int elemento;
        j--;
        elemento = buffer[j];
        consumir_dato (elemento) ;
        sem_post(&hay_lugar);
    }
    return NULL ;
}

int main()
{
    pthread_t tidprod,tidcons;

```

```
pthread_mutex_init(&mutex, NULL);
sem_init(&hay_lugar, 0, 5);
sem_init(&hay_datos, 0, 0);

pthread_create(&tidprod, NULL, productor, NULL);
pthread_create(&tidcons, NULL, consumidor, NULL);
pthread_join(tidprod, NULL);
pthread_join(tidcons, NULL);

return 0;
}
```

Comentario:

Pregunta 5

Finalizado

Puntúa 1,50 sobre 1,50

Marcar pregunta

En un sistema Unix, como GNU/Linux, que cuenta con un servidor de aplicaciones Apache contamos con un archivo de registro de accesos en `/var/log/apache2/access_log` dentro donde va almacenando un registro de las operaciones solicitadas al servidor, que normalmente consistirán en la petición de un archivo.

Cada línea contiene:

- La dirección IP desde donde ha solicitado el servicio . (Ej: 172.21.128.70).
- La fecha y hora en formato GMT en la que se solicitó el servicio. (Ej: [21/May/2020:11:22:05 -0300])
- La petición realizada al servidor entre comillas (Ej.: "GET/app1/lib/yuilib/3.17.2/json-stringify/json-stringify-min.js HTTP/1.1"). Cada petición consta de un comando, en este caso GET; un recurso, en éste caso/app1/lib/yuilib/3.17.2/json-stringify/json-stringify-min.js; y la versión del protocolo http con la que se trabaja ,en éste caso 1.1.
- Un código de tres dígitos numéricos que indica el estado de la operación realizada. El código 200, indica que la operación se ha realizado con éxito. Un código 4xx o 5xx, indica que ha habido algún error en la operación.
- El número de bytes que se han transferido.

A partir del archivo de `access_log` que se presenta a continuación:

```
172.16.3.8 - - [20/Feb/2020:06:38:37 -0300] "GET /error/OpenSans-Semibold.ttf HTTP/1.1" 404 359 "http://www.misitio.com.ar/error-test/codes.css" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.116 Safari/537.36"
172.16.3.8 - - [20/Feb/2020:07:02:14 -0300] "GET / HTTP/1.1" 504 543 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.116 Safari/537.36"
172.16.3.8 - - [20/Feb/2020:07:02:33 -0300] "GET /error/OpenSans-Semibold.ttf HTTP/1.1" 499 0 "http://www.misitio.com.ar/error-test/codes.css" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.116 Safari/537.36"
172.16.0.78 - - [20/Feb/2020:07:44:00 -0300] "GET /error-test/codes.css HTTP/1.1" 200 526 "http://www.misitio.com.ar/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/76.0.3809.132 Safari/537.36"
172.16.0.78 - - [20/Feb/2020:07:44:00 -0300] "GET /error-test/bgpattern.png HTTP/1.1" 200 4248 "http://www.misitio.com.ar/error-test/codes.css" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/76.0.3809.132 Safari/537.36"
172.16.0.78 - - [20/Feb/2020:07:44:00 -0300] "GET /favicon.ico HTTP/1.1" 404 359 "http://www.misitio.com.ar/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/76.0.3809.132 Safari/537.36"
```

Indique las acciones que realiza la siguiente secuencia de comandos y el resultado espera para el archivo de log presentado:

- `grep -e '[ ][5][0-9][0-9][ ]' access.log | grep wc -l`
- `cat access.log | grep app1 > salida.log`

a) devuelve la cantidad de líneas que cumplen con el patrón de tener un 5, seguido de un numero del 0 al 9, seguido de un numero del 0 al 9

b) guarda en un archivo llamado salida.log las líneas de access.log que contengan la cadena app1 en este caso no hay líneas que cumplan esa condición con lo cual el archivo quedará vacío

Comentario:

Finalizar revisión

Navegación por el cuestionario

- 1
- 2
- 3
- 4
- 5

Finalizar revisión